

1. Datatypes

In C programming, a data type is a classification that tells the compiler how much memory space to allocate for a variable and what kind of values (integers, decimals, characters) that variable can store.

- **char:** Used to store a single text character or symbol (like 'A', '7', or '\$'). It occupies 1 byte of memory and internally holds the character's numeric ASCII value.
- **int:** Used to store whole numbers without any decimals (like 42 or -100). It typically occupies 4 bytes of memory on modern systems.
- **float:** Used to store fractional or decimal numbers (like 3.14 or -0.005). It provides single-precision accuracy and occupies 4 bytes.
- **double:** Used for decimal numbers requiring high precision. It provides double the precision of a float and occupies 8 bytes.
- **long:** Used to store larger whole numbers than a standard int. Depending on the operating system (32-bit vs. 64-bit), it takes up either 4 or 8 bytes.
- **unsigned int:** A variation of int that only stores positive whole numbers (and zero). By dropping negative numbers, it doubles the maximum positive value it can hold within its 4 bytes.
- **long long:** Used to store extremely large whole numbers. It guarantees at least 8 bytes (64 bits) of space across all modern systems.

```
#include <stdio.h>

int main() {
    printf("Size of char: %zu byte(s)\n", sizeof(char));
    printf("Size of int: %zu byte(s)\n", sizeof(int));
    printf("Size of float: %zu byte(s)\n", sizeof(float));
    printf("Size of double: %zu byte(s)\n", sizeof(double));
    printf("Size of long: %zu byte(s)\n", sizeof(long));
    printf("Size of unsigned int: %zu byte(s)\n", sizeof(unsigned int));
    printf("Size of long long: %zu byte(s)\n", sizeof(long long));
    return 0;
}
```

```
[root@2f629f9c91ab:/home/ubuntu/class# gcc size.c -o size
[root@2f629f9c91ab:/home/ubuntu/class# ./size
Size of char: 1 byte(s)
Size of int: 4 byte(s)
Size of float: 4 byte(s)
Size of double: 8 byte(s)
Size of long: 8 byte(s)
Size of unsigned int: 4 byte(s)
Size of long long: 8 byte(s)
```

Fig 1: Size of Datatypes

Memory Segment Address

When an operating system runs a compiled C program, it creates a process and allocates a structured block of virtual memory. This memory is divided into distinct zones called segments, each designated for a specific type of variable or data.

- **Data Segment:** Stores global and static variables that are explicitly initialized in the source code (e.g., `int x = 42;`). It resides in a low, fixed-size memory region.
- **BSS Segment:** Stores global and static variables that are uninitialized or initialized to zero (e.g., `int y;`). The OS automatically clears this memory to zero before execution.
- **Heap Segment:** Used for dynamic memory allocation requested at runtime using functions like `malloc()`, `calloc()`, or `realloc()`. It starts at lower addresses right after the BSS segment and grows upward toward higher addresses.
- **Stack Segment:** Used for automatic variables, including local functions variables (e.g., `int local_var;`) and function arguments. It sits at the very highest part of memory and grows downward toward the heap as functions are called.

```
#include <stdio.h>
#include <stdlib.h>

int global = 42;
int uninitialized;

int main() {
    int local_stack_var = 10;
    int *heap_var = (int *)malloc(sizeof(int));
    if (heap_var == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    printf("Data Segment (Global Initialized) : %p\n", (void*)&global);
    printf("BSS Segment (Global Uninitialized) : %p\n", (void*)&uninitialized);
    printf("Heap Segment (Dynamic Allocation) : %p\n", (void*)heap_var);
    printf("Stack Segment (Local Variable) : %p\n", (void*)&local_stack_var);
    long long difference = (char*)&local_stack_var - (char*)heap_var;
    printf("Address difference (Stack - Heap) : %lld bytes\n", difference);

    free(heap_var);
    return 0;
}
```

Fig 2: Source code for memory segment

```
[root@2f629f9c91ab:/home/ubuntu/class# ./memory
Data Segment (Global Initialized) : 0xaaaade6a0010
BSS Segment (Global Uninitialized) : 0xaaaade6a0018
Heap Segment (Dynamic Allocation) : 0xaaab152b32a0
Stack Segment (Local Variable) : 0xfffffe2e89834
Address difference (Stack - Heap) : 93822717355412 bytes
```

Fig 3: Output

Knowledge Check

1. What is a data type?

In programming and operating systems, a data type is a blueprint that tells the compiler and CPU how much binary memory space to allocate for a variable and how to interpret those raw bits (e.g., treating them as text, whole integers, or decimal fractions).

2. Five Main Memory Segments

The five primary segments in a standard process memory layout are:

- Text (Code) Segment
- Data Segment (Initialized data)
- BSS Segment (Uninitialized data)
- Heap Segment (Dynamic allocations)
- Stack Segment (Local runtime frame)

3. Data Segment vs. BSS Segment

The Data segment stores global and static variables that have been explicitly assigned a value by the programmer (e.g., `int x = 5;`), whereas the BSS segment stores global and static variables that are uninitialized or zero-filled (e.g., `int y;`), which the OS clears to zero right before execution starts.

4. Why Stack is High and Text is Low

The Text segment is placed at low, fixed virtual addresses because its size is constant and read-only, preventing it from expanding. The Stack is placed at the highest addresses because it needs to grow downward dynamically into a vast pool of unallocated memory toward the upward-growing heap, ensuring the two regions don't easily crash into each other.

5. Stack Behavior After a Function Returns

When a function returns, its specific memory frame becomes invalid because the CPU simply moves the Stack Pointer (SP) register back up to the previous frame's address. The OS doesn't actively clear it; instead, the CPU architecture reclaims it implicitly by allowing subsequent function calls to overwrite that exact memory space.

6. How the OS Isolates Process Memory

The OS keeps track of process boundaries using Virtual Memory and a hardware-driven system called Paging. Every individual process has its own unique Page Table managed by the kernel; when a process requests a memory address, the hardware Memory Management Unit (MMU) translates that virtual request into an isolated physical RAM block, ensuring one program physically cannot access or modify the memory maps of another.

Github Link: [*https://github.com/abhishekk1004/os*](https://github.com/abhishekk1004/os)